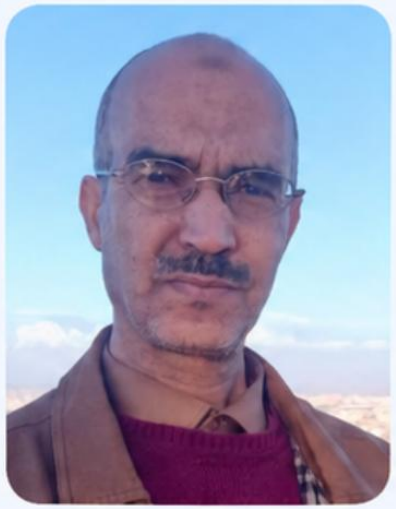
Algorithmique avancée et complexité

Équipe pédagogique

Enseignants de la matière

La matière est assurée par une équipe pédagogique composée d'un enseignant responsable du cours et des TD, et d'un enseignant responsable du TP.



Dr. Benyamina Ahmed

Responsable du cours et des TD

Cours
Présentation du cours, concepts et fondements.

TD
Encadrement des travaux dirigés et résolution d'exercices.



Mahkouka Mohamed Fouad

Responsable du TP (Doctorant)

TP
Encadrement des travaux pratiques et mise en œuvre des algorithmes.

Composantes de la matière

Cours | TD | TP

Coefficient
3

Crédits
5

Spécialités concernées

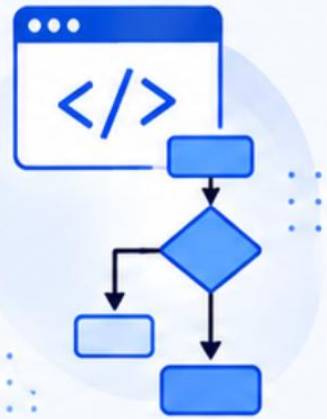
1. Intelligence Artificielle
2. Réseaux et Systèmes Distribués

Programme de la matière Algorithmique avancée et complexité

01

Chapitre 1

Introduction à la complexité algorithmique

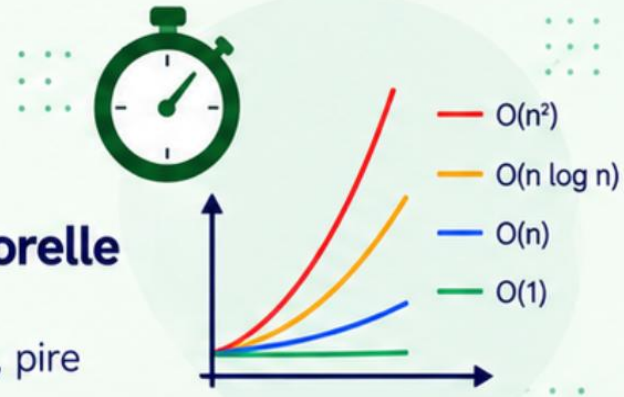


Notions préliminaires, efficacité des algorithmes et mesure de la complexité.

02

Chapitre 2

Analyse de la complexité temporelle

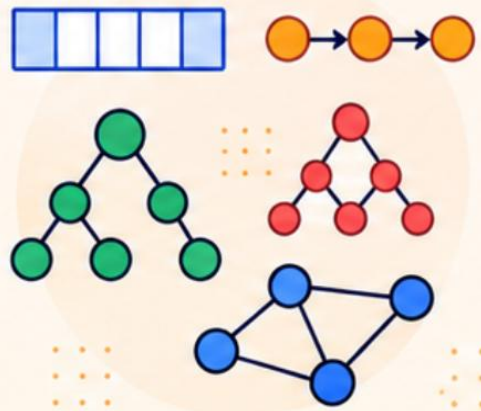


Complexité temporelle, meilleur, pire et moyen cas, et notation O.

03

Chapitre 3

Structures de données et complexité

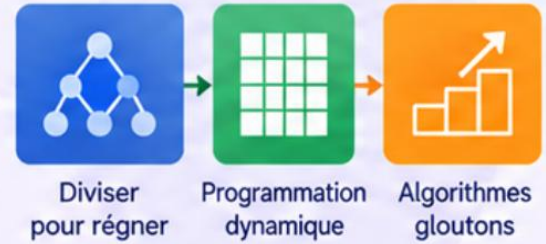


Tableaux, listes, arbres, tas et graphes, et leur impact sur les performances.

04

Chapitre 4

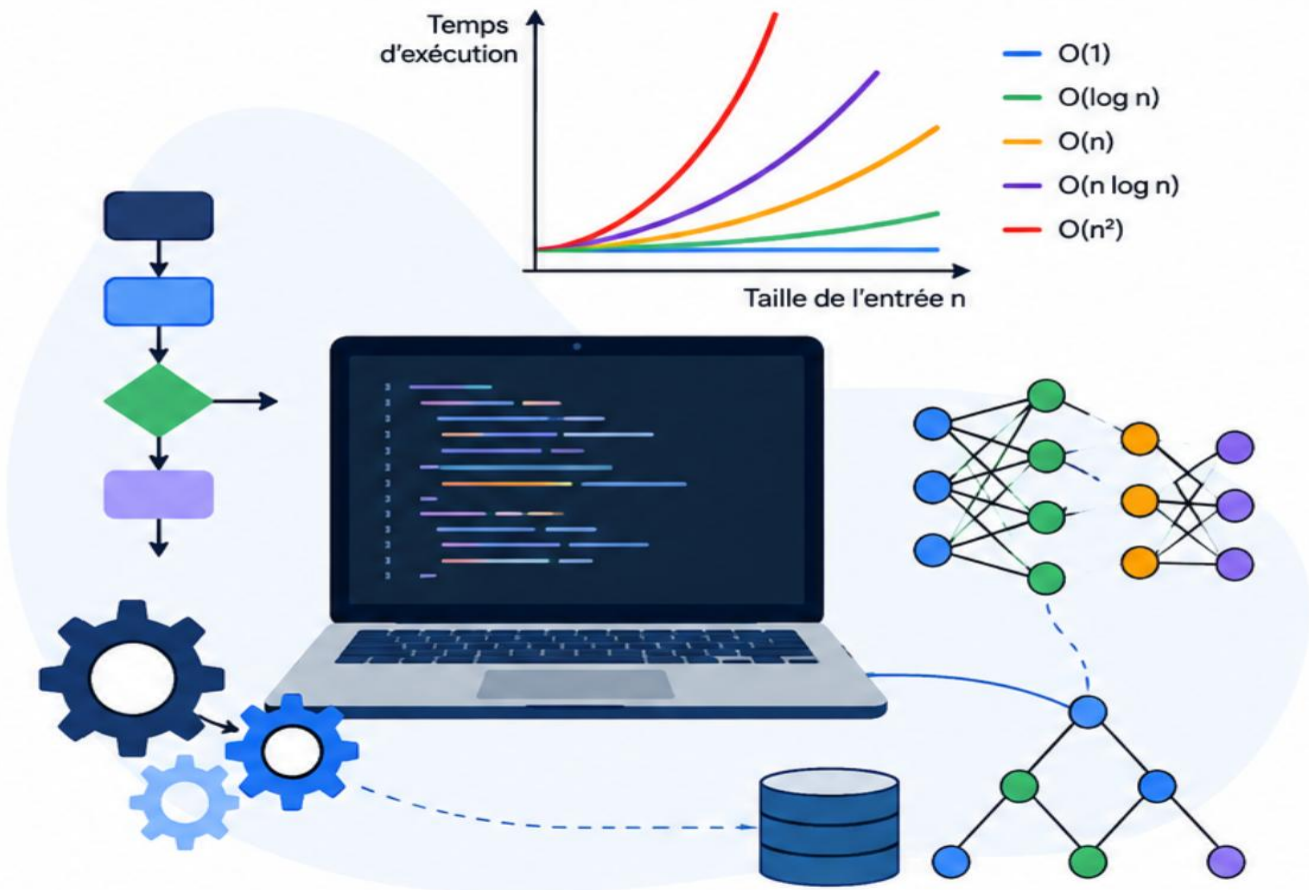
Techniques d'analyse de complexité



Diviser pour régner, programmation dynamique et algorithmes gloutons.

Introduction à la complexité algorithmique

Notions préliminaires



CHAPITRE 1 • COURS

1. Motivation

Traitement d'une quantité croissante de données n.

Un algorithme peut donner la bonne réponse, mais son comportement dépend de la quantité de données.

Quand cette quantité augmente, le **temps** d'exécution et la **mémoire** utilisée peuvent devenir très grands.



Objectif : Réaliser la complexité d'un algorithme en **temps** et en **mémoire** afin de prévoir ses performances et choisir une solution adaptée.

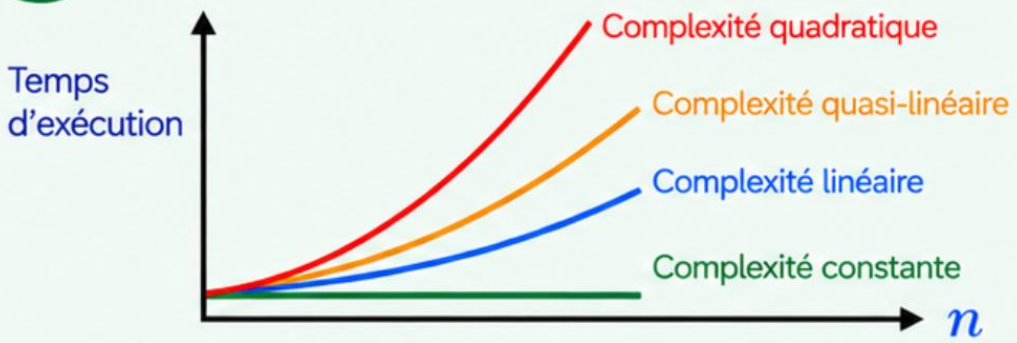
1. Motivation

Pourquoi c'est important ?

Un algorithme peut donner la bonne réponse, mais son comportement dépend de la quantité de données. Quand cette quantité augmente, le **temps** d'exécution et la **mémoire** utilisée peuvent devenir très grands.

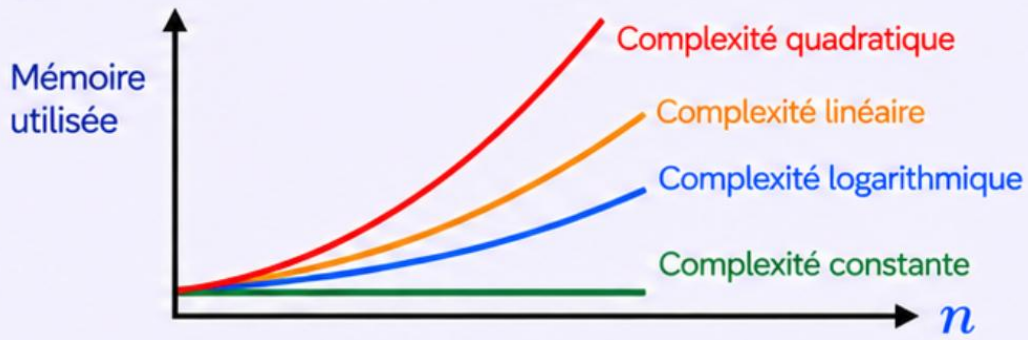
Complexité temporelle

Temps d'exécution nécessaire en fonction de n.



Complexité spatiale

Mémoire utilisée nécessaire en fonction de n.



Objectifs :

- Prévoir le comportement d'un algorithme quand n augmente.
- Comparer plusieurs algorithmes pour le même problème.
- Choisir la solution la plus adaptée aux ressources disponibles (temps, mémoire).

CHAPITRE 1 • COURS

Qu'est-ce qu'un algorithme ?

Un algorithme est une suite finie et non ambiguë d'instructions permettant de **résoudre un problème**.

Définition

Un algorithme est :

Fini
Il se termine après un nombre fini d'étapes.

Défini
Chaque étape est précise et sans ambiguïté.

À entrées
Il prend zéro ou plusieurs entrées.

À sorties
Il produit au moins une sortie.

Effectif
Chaque étape est élémentaire et réalisable.

Remarque : Un algorithme n'est pas lié à un langage de programmation. Il décrit une méthode, indépendante de toute implémentation.

CHAPITRE 1 • COURS

L'idée clé d'un algorithme

Un algorithme transforme des entrées en sorties en suivant des règles claires et ordonnées.



Données d'entrée
(zéro ou plusieurs)

Algorithme
Règles claires et ordonnées

Données de sortie
(au moins une)

Idée clé

Un algorithme transforme des entrées en sorties
en suivant des règles claires et ordonnées.

CHAPITRE 1 • COURS

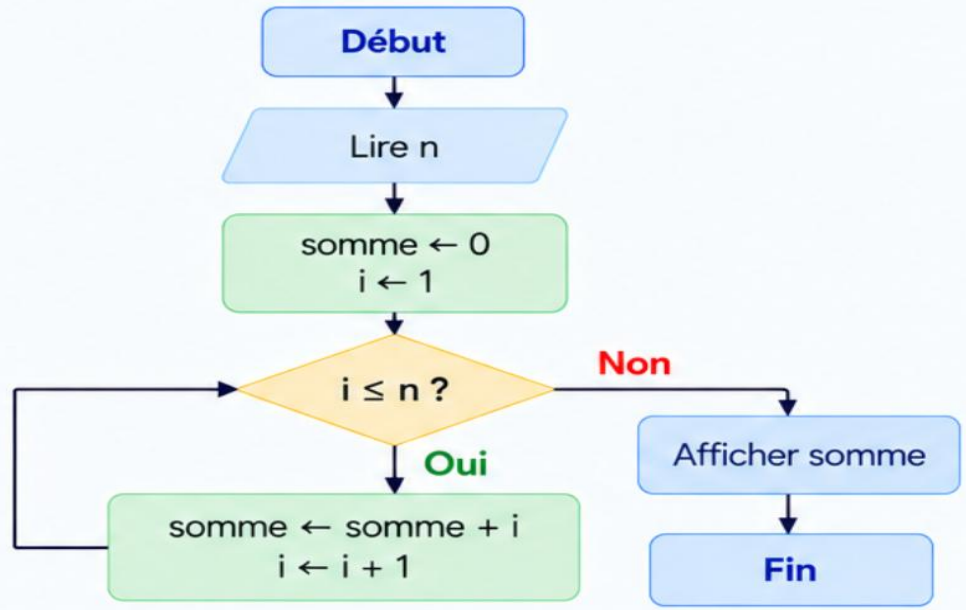
Exemple : calcul de la somme de n entiers

Calculer la somme des n premiers entiers positifs.

Pseudo-code

```
1  Entrée : n (entier positif)
2  somme ← 0
3  i ← 1
4  Tant que i ≤ n faire
5      somme ← somme + i
6      i ← i + 1
7  Fin Tant que
8  Sortie : somme
```

Organigramme



Interprétation

- On initialise la somme à 0 et l'indice à 1.
- À chaque itération, on ajoute i à la somme et on incrémente i.
- Lorsque i devient strictement supérieur à n, on affiche la somme.

Résultat

À la fin, la variable somme contient :

$$1 + 2 + \cdots + n.$$

CHAPITRE 1 · COURS

Qu'est-ce qu'un problème algorithmique ?

Un problème algorithmique est une question claire à laquelle on cherche **une solution** à l'aide d'un algorithme.

Définition

Un problème algorithmique est défini par :

Des entrées
Les données fournies au problème.

Une question
Ce que l'on cherche à déterminer.

Des sorties attendues
Les résultats corrects pour toutes les entrées valides.

Des contraintes
Ressources limitées (temps, mémoire, espace, etc.).

Idée clé

Pour chaque problème, il peut exister plusieurs algorithmes possibles, avec des performances différentes.



Remarque : Bien formuler un problème est la première étape essentielle avant de concevoir un algorithme efficace.

CHAPITRE 1 · COURS

Les entrées et la question

Un problème algorithmique est défini par des entrées et une question à déterminer.

Des entrées

Les entrées sont les données fournies au problème.



Exemples d'entrées :

- un entier n
- une liste L
- un arbre
- une valeur x

Une question

La question est ce que l'on cherche à déterminer.



Exemples de questions :

- Calculer la somme des n entiers.
- Rechercher un élément dans une liste.
- Trier une liste.
- Parcourir un arbre.

Remarque : Bien formuler un problème est la première étape essentielle avant de concevoir un algorithme efficace.

CHAPITRE 1 • COURS

Les sorties attendues et les contraintes

Un problème algorithmique spécifie les résultats attendus et les contraintes à respecter.

Des sorties attendues

Les sorties sont les résultats corrects pour toutes les entrées valides.



Exemples de sorties :

- un entier (résultat d'un calcul)
- une valeur x (élément trouvé)
- la liste L triée
- un ordre de visite des nœuds

Des contraintes

Un problème algorithmique peut être soumis à des contraintes.

Temps
Temps d'exécution limité.

Mémoire
Mémoire limitée.

Espace
Espace de stockage limité.

Autres
Etc.

Remarque : Bien formuler un problème est la première étape essentielle avant de concevoir un algorithme efficace.

CHAPITRE 1 • COURS

Exemples de problèmes algorithmiques

Pour chaque problème, il peut exister plusieurs algorithmes possibles, avec des performances différentes.

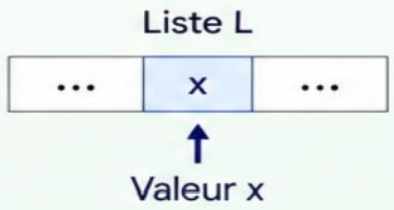
Somme des n entiers

Entrée : n entiers
Sortie : leur somme

$$\sum_{i=1}^{n} i$$

Recherche d'un élément

Entrée : une liste L et une valeur x
Sortie : l'indice de x dans L ou −1



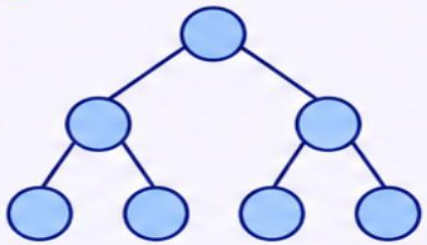
Tri d'une liste

Entrée : une liste L
Sortie : la liste L triée



Parcours d'un arbre

Entrée : un arbre
Sortie : un ordre de visite des nœuds



Remarque : Bien formuler un problème est la première étape essentielle avant de concevoir un algorithme efficace.

SECTION 2 · DÉFINITIONS

Exemple : un même problème, plusieurs algorithmes possibles

Il peut exister plusieurs façons de résoudre le même problème, avec des performances différentes.

Problème

Trier une liste de n nombres
Entrée : une liste de n nombres
Sortie : la liste triée par ordre croissant

Plusieurs algorithmes possibles

Tri par insertion

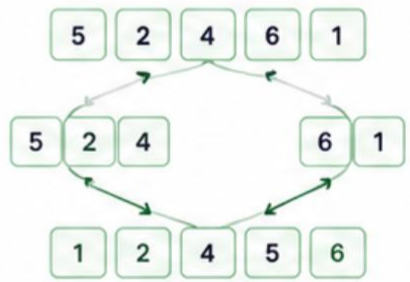
Construit la liste triée en insérant chaque élément à sa place.



Temps : lent pour grandes listes
Mémoire : faible

Tri fusion (Merge Sort)

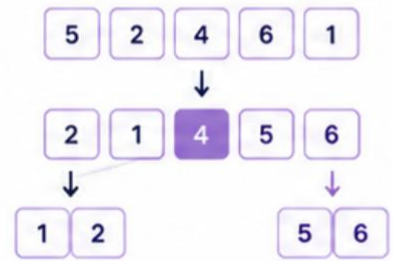
Divise la liste en deux, trie récursivement, puis fusionne.



Temps : efficace et prévisible
Mémoire : nécessite plus d'espace

Tri rapide (Quick Sort)

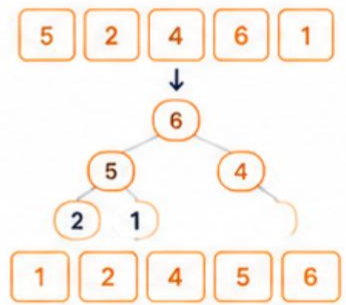
Choisit un pivot, partitionne, puis applique récursivement.



Temps : très rapide en moyenne
Mémoire : faible (en général)

Tri par tas (Heap Sort)

Construit un tas, puis extrait successivement le maximum.



Temps : efficace et prévisible
Mémoire : faible

Idée clé : plusieurs algorithmes peuvent résoudre le même problème, **mais leurs coûts (temps et mémoire) varient**.
L'analyse de complexité permet de **choisir le plus adapté** selon le contexte.

CHAPITRE 1 • COURS

Les caractéristiques d'un bon algorithme

Un bon algorithme doit respecter un ensemble de propriétés essentielles qui garantissent sa **pertinence**, son **efficacité** et sa fiabilité.



Correction

L'algorithme doit produire la bonne sortie pour toutes les entrées valides.



Finitude

L'algorithme doit se terminer après un nombre fini d'étapes.



Efficacité

Il doit utiliser raisonnablement le temps d'exécution et la mémoire.



Simplicité

Il doit être simple à comprendre, à implémenter et à maintenir.



Robustesse

Il doit fonctionner correctement même en présence de cas particuliers.



Généralité

Il doit pouvoir être appliqué à une large classe de problèmes.



Idée clé — Un algorithme n'est pas seulement correct : il doit aussi être **efficace, simple** et **robuste.**

CHAPITRE 1 • COURS

Correction et finitude

Deux propriétés essentielles qu'un bon algorithme doit respecter.

Correction

L'algorithme doit produire la bonne sortie pour toutes les entrées valides.

Exemple : tri d'une liste

Entrée (liste de n nombres)

5	2	4	6	1

→

Sortie (liste triée)

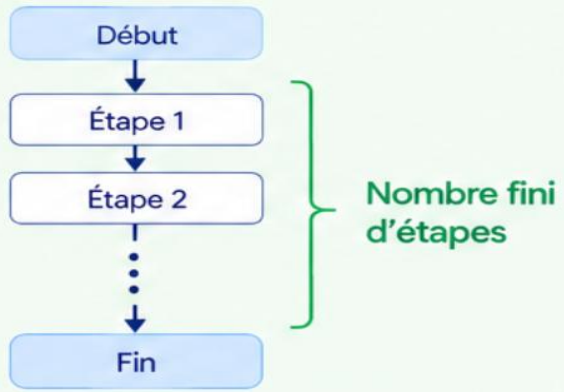
1	2	4	5	6

L'algorithme est **correct** s'il produit la sortie attendue pour toutes les entrées valides.

Finitude

L'algorithme doit se terminer après un nombre fini d'étapes.

Idée



L'algorithme se termine toujours en un nombre fini d'étapes.

Remarque : La **correction** garantit la bonne sortie, tandis que la **finitude** assure que l'algorithme termine.

CHAPITRE 1 • COURS

Efficacité, simplicité, robustesse et généralité

Quatre autres propriétés essentielles qu'un bon algorithme doit respecter.

Efficacité

Il doit utiliser raisonnablement le temps d'exécution et la mémoire.

- Temps d'exécution raisonnable
- Mémoire raisonnable

Simplicité

Il doit être simple à comprendre, à implémenter et à maintenir.

Facile à comprendre → Facile à implémenter → Facile à maintenir

Robustesse

Il doit fonctionner correctement même en présence de cas particuliers.

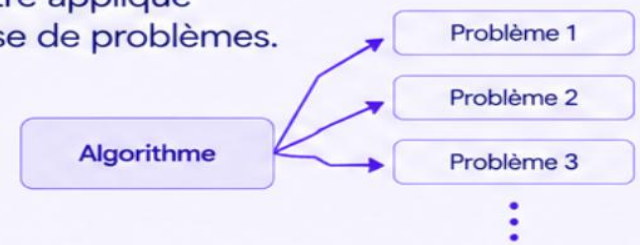
Exemples de cas particuliers

- Entrées limites
- Données particulières
- Cas d'erreurs

Généralité

Il doit pouvoir être appliqué à une large classe de problèmes.

Un même algorithme peut être réutilisé dans différents contextes.



Idée clé : Un bon algorithme n'est pas seulement correct : il doit aussi être **efficace, simple** et **robuste.**

SECTION 3 • PRÉLIMINAIRES

Les structures de données de base

Idée clé
Différentes structures organisent les données différemment → **coûts différents.**

Le choix de la structure de données influence directement le **temps d'éxécution** et la **mémoire** utilisée.

Tableau

Éléments stockés de façon **contiguë**

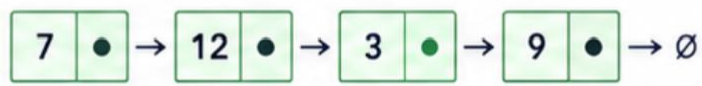


Temps
Accès direct à un élément

Mémoire
Taille fixée à l'avance

Liste chaînée

Éléments (nœuds) liés par des **pointeurs**



Temps
Parcours séquentiel

Mémoire
Taille dynamique

Arbre & graphe

Relations **hiérarchiques** ou **connexes**



Temps
Pecherche selon la structure

Mémoire
Dépend du nombre de liens

Même problème, structures différentes → **temps d'éxécution** et **mémoire** peuvent varier considérablement.

SECTION 4 • ANALYSE

Taille de l'entrée, pire cas, meilleur cas, cas moyen

Pour analyser un algorithme, on compte le **nombre d'opérations** élémentaires effectuées.
Ce coût dépend de la **taille de l'entrée (n)** et du **cas considéré**.

Idée clé

Un même algorithme peut avoir des coûts très différents selon l'entrée. On analyse trois cas : **pire cas**, **meilleur cas** et **cas moyen** pour comprendre son comportement.

1. Taille de l'entrée

Le paramètre *n* caractérise la taille du problème.

Tableau
n = nombre d'éléments

Liste chaînée
n = nombre de nœuds

Arbre / Graphe
n = nombre de nœuds et/ou arêtes

2. Pire cas

Nombre **maximal** d'opérations sur les **entrées** de taille *n*.

Garantie de performance

Valable pour toutes les entrées.

Ex. recherche : *n* comparaisons.

3. Meilleur cas

Nombre **minimal** d'opérations sur les entrées de taille *n*.

Rare mais informatif

Donne une borne inférieure.

Ex. recherche : 1 comparaison.

4. Cas moyen

Moyenne du nombre d'opérations sur **toutes les entrées** de taille ***n*** selon une distribution.

Dépend d'un modèle de probabilités.

Ex. recherche : n/2 comparaisons.

Objectif de l'analyse : comparer ces trois cas pour choisir la solution la plus adaptée (**temps**, **mémoire**, **ressources**).

SECTION 4 • ANALYSE

1. Taille de l'entrée

Le paramètre *n* caractérise la taille du problème.

Idée clé

Le coût d'un algorithme dépend de la **taille de l'entrée (n)** et du **cas considéré**.

Exemples de structures d'entrée

Tableau

n = nombre d'éléments



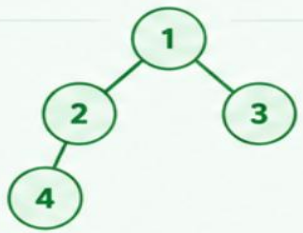
Liste chaînée

n = nombre de nœuds



Arbre / Graphe

n = nombre de nœuds et/ou arêtes



Rôle de *n*

Le paramètre *n* caractérise la **taille du problème**.

↓

On compte le **nombre d'opérations élémentaires** effectuées.

↓

Ce coût dépend de la **taille de l'entrée (n)** et du **cas considéré**.

Remarque : La taille de l'entrée permet de comparer les performances des algorithmes de manière cohérente.

SECTION 4 • ANALYSE

2. Pire cas, meilleur cas et cas moyen

On analyse un algorithme selon le **nombre d'opérations** élémentaires effectuées sur les entrées de taille n.

Idée clé

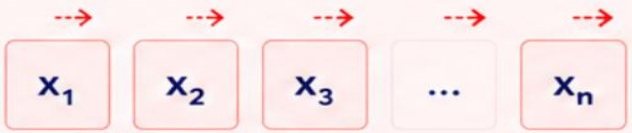
Un même algorithme peut avoir des coûts très différents selon l'entrée. On analyse trois cas : **pire cas**, **meilleur cas** et **cas moyen** pour comprendre son comportement.

Pire cas

Nombre **maximal** d'opérations sur les **entrées** de taille n.

Garantie de performance

Valable pour toutes les entrées.
Ex. recherche : n comparaisons.



Meilleur cas

Nombre **minimal** d'opérations sur les entrées de taille n.

Rare mais informatif

Donne une borne inférieure.
Ex. recherche : 1 comparaison.



Cas moyen

Moyenne du nombre d'opérations sur **toutes les entrées** de taille n selon une distribution.

Dépend d'un modèle de probabilités.

Ex. recherche : $n/2$ comparaisons.



Remarque : Ces trois analyses permettent de comparer des algorithmes et de choisir la solution la plus adaptée (**temps**, **mémoire**, **ressources**).

SECTION 4 • ANALYSE

3. Exemple — recherche de *v* dans T (n éléments)

On illustre les trois cas d'analyse (meilleur cas, pire cas et cas moyen) sur l'exemple de la recherche d'un élément *v* dans une structure de taille *n*.

Idée clé

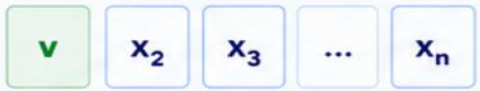
Un même algorithme peut avoir des coûts très différents selon l'entrée. On analyse trois cas : **pire cas**, **meilleur cas** et **cas moyen** pour comprendre son comportement.

Recherche de *v* dans T (n éléments)

Meilleur cas

v en première position

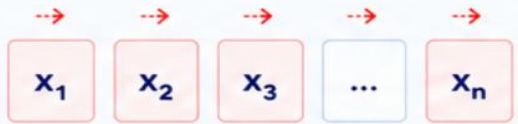
1 comparaison



Pire cas

v absente de T

n comparaisons



Cas moyen (ex. recherche linéaire)

v à une position aléatoire

≈ **n/2** comparaisons



Même *n*, coûts opposés — il faut toujours **préciser le cas analysé.**

Ces trois cas permettent de comparer des algorithmes et de choisir la solution la plus adaptée (**temps**, **mémoire**, **ressources**).

On retient le pire cas :

- C'est une **garantie** de performance pour toute entrée, et il est souvent **proche du cas moyen**.
- Ces trois analyses permettent de comparer des algorithmes et de choisir la solution la plus adaptée (**temps**, **mémoire**, **ressources**).

SECTION 4 • ANALYSE

Pseudo-code annoté : l'algorithme du maximum

On considère l'algorithme MAXIMUM(T) avec le comptage des comparaisons et des affectations.

PSEUDO-CODE ANNOTÉ

```
1   MAXIMUM(T)                  # n = longueur(T)
2     max ← T[1]                # 1 affectation
3     pour i ← 2 à n faire      # n − 1 itérations
4         si T[i] > max         # n − 1 comparaisons
5             max ← T[i]        # ≤ n − 1 affectations
6     retourner max
```

1 Ligne 1 : définition de l'instance
n = longueur(T)

2 Ligne 2 : initialisation
On met max ← T[1].
→ 1 affectation.

3 Ligne 3 : boucle
On répète pour i ← 2 à n.
→ $n - 1$ itérations.

4 Ligne 4 : comparaison
On teste si T[i] > max.
→ $n - 1$ comparaisons.

5 Ligne 5 : mise à jour
Si T[i] > max, on fait max ← T[i].
→ $\leq n - 1$ affectations.

6 Ligne 6 : retour du résultat
On retourne max.

Idée clé

On compte les opérations ligne par ligne sur une instance T de taille n.
Le nombre total d'opérations est :
$T(n) = 2n - 1$.

Types d'opérations

Comparaison
Test du type T[i] > max.

Affectation
Mise à jour d'une variable.
Ex. : max ← T[i].

Bilan (depuis le pseudo-code)

Nombre d'itérations : $n - 1$
Nombre de comparaisons : $n - 1$
Nombre d'affectations : $\leq n - 1$

Remarque : Le nombre total d'opérations élémentaires est $T(n) = 2n - 1$.

SECTION 4 • ANALYSE

Analyse des trois cas : pire cas, meilleur cas et cas moyen

On étudie le **nombre d'opérations** selon le comportement de l'algorithme sur les entrées de taille n.

Pire cas

Le maximum est à la dernière position ou le tableau est décroissant.

Nombre de comparaisons : $n-1$

Nombre d'affectations : $n-1$

Exemple de cas

Tableau décroissant :

5	4	3	2	1	...

$n-1$ comparaisons et $n-1$ affectations.

Meilleur cas

Le premier élément est déjà le maximum.

Nombre de comparaisons : 1

Nombre d'affectations : 0

Exemple de cas

Le maximum est en première position :

5	2	1	3	4	...

1 comparaison et 0 affectation.

Cas moyen

La position du maximum suit une distribution uniforme (toutes les positions équiprobables).

Nombre de comparaisons : $\approx n/2$

Nombre d'affectations : $\approx n/2$

Exemple (interprétation)

Le maximum est en moyenne à une position aléatoire parmi les n positions.

2	5	1	4	3	...

$\approx n/2$ comparaisons et $\approx n/2$ affectations.

Ces trois analyses permettent de comparer des algorithmes et de choisir la solution la plus adaptée (**temps**, **mémoire**, **ressources**).

SECTION 4 • ANALYSE

Exemple et vérification : recherche du maximum

On vérifie le déroulement de l'algorithme sur une instance T et on confirme le nombre total d'opérations.

Vérification sur $T = [4, 2, 5, 1, 3]$ $(n = 5)$

i	T[i]	max (après)	Comparaison	Affectation max
—	—	4	initialisation	1
2	2	4	2 > 4 ? non ✗	0
3	5	5	5 > 4 ? oui ✓	1
4	1	5	1 > 5 ? non ✗	0
5	3	5	3 > 5 ? non ✗	0

Comparaisons : $n - 1 = 4$

Affectations : $1 + (\leq n - 1) = \leq 5$

Pour $n = 5$: $2n - 1 = 2 \times 5 - 1 = 9$ opérations → confirmé par le déroulement.

Idée clé

On compte les opérations ligne par ligne sur une instance T de taille n.
Dans cet algorithme, le nombre total d'opérations est : $T(n) = 2n - 1$.

Méthode générale

1. Identifier n.
2. Compter les opérations de chaque ligne.
3. Multiplier par le nombre d'exécutions.
4. Additionner toutes les contributions.

On retient le pire cas :

- c'est une **garantie** de performance pour toute entrée, et il est souvent **proche du cas moyen**.
- Ces trois analyses permettent de comparer des algorithmes et de choisir la solution la plus adaptée (**temps**, **mémoire**, **ressources**).